



Playbook和handler用法

誉天教育

- Ansible Playbook简介
- YAML基本语法
- Playbook基本语法
- Playbook简单示例
- ansible-playbook常用选项
- Multiple Plays
- playbook的结构说明
- handler用法

- ansible-playbook是一系列ansible命令的集合，利用yaml语言编写。playbook命令根据自上而下的顺序依次执行。同时，playbook开创了很多特性，它可以允许你传输某个命令的状态到后面的指令，如你可以从一台机器的文件中抓取内容并附为变量，然后在另一台机器中使用，这使得你可以实现一些复杂的部署机制，这是ansible命令无法实现的。
- playbook通过ansible-playbook命令使用，它的参数和ansible命令类似，如参数-k(-ask-pass) 和 -K (-ask-sudo) 来询问ssh密码和sudo密码，-u指定用户，这些指令也可以通过规定的单元写在playbook。ansible-playbook的简单使用方法: ansible-playbook example-play.yml 。

- YAML 是专门用来写配置文件的语言，非常简洁和强大，远比 JSON 格式方便。YAML 语言的设计目标，就是方便人类读写。
- YAML Ain't Markup Language，即YAML不是XML。不过，在开发的这种语言时，YAML的意思其实是：“Yet Another Markup Language”（仍是一种标记语言）。它的基本语法规则如下：
 - ◆ 大小写敏感
 - ◆ 使用缩进表示层级关系
 - ◆ 缩进时不允许使用Tab键，只允许使用空格。
 - ◆ 缩进的空格数目不重要，只要相同层级的元素左侧对齐即可
 - ◆ # 表示注释，从这个字符一直到行尾。

- 支持数据类型

1、纯量 (scalars) : 单个的、不可再分的值

数据最小的单位，不可以再分割，类似于Python中单个变量。

2、数组：一组按次序排列的值，又称为序列 (sequence) / 列表 (list)

与Python的list数组结构类似，数组元素使用短横线 "-" 开头。

- Jack

也可以写成一行

- Harry

[Jack,Harry,Sunny]

- Sunny

对应到python的list写法如下：

['Jack','Harry','Sunny']

3、对象：键值对的集合，又称为映射（mapping）/ 哈希（hashes）/ 字典（dictionary）

对象的一组键值对，使用冒号结构表示。类似Python中的字典数据结构。

```
platformName: Android
```

```
platformVersion: 6.0.1
```

Yaml 也允许另一种写法，将所有键值对写成一个行内对象。

```
{platformName: Android,platformVersion: 6.0.1}
```

注意：冒号后面一定要有空格！对应到python字典的写法如下：

```
{'platformName': 'Android', 'platformVersion': '6.0.1'}
```

举例:

name: Tom

age: 27

wife:

name: Jerry

age: 25

children:

- name: Jack

age: 15

- name: Bob

age: 14

解释:

(1)最外层的name和age和wife和children组成的是对象--即类似字典数据结构

(2)wife里面的name和age组成的是对象(即类似字典); wife又和这个类似字典的再在一起组成对象即python中写法为{'wife':{'name':'Jerry','age':25}}

wife:

name: Jerry

age: 25

(3)这个

- name: Jack

age: 15

和这个组成一个数组: 因为是用短横线开头的; 然后两者合起来和children组成一个对象(即类似字典)

- name: Bob

age: 14

即python中写法为 [{'name':'Jerry','age':25},{'name':'Jack','age':14}]

即python中children对象写法为: {'children':[{'name':'Jerry','age':25},{'name':'Jack','age':14}]}

- 下面是一个简单的ansible-playbook示例，可以了解其构成:

```
# cat user.yml
- name: create user
  hosts: all
  remote_user: root
  gather_facts: false
  vars:
    user: "test"
  tasks:
    - name: create user
      user: name="{{ user }}"
```

配置项说明：

name：对该playbook实现的功能做一个概述，后面执行过程中，会打印 name变量的值

hosts：指定对哪些被管理机进行操作；

remote_user：指定在远程被管理机上执行操作时使用什么用户，如不指定，则使用ansible.cfg中配置的remote_user

gather_facts：指定在执行任务之前，是否先执行setup模块获取主机相关信息，如未用到，可不指定

vars：定义后续任务中会使用到的变量，如未用到，可不指定

tasks：定义具体需要执行的任务

name：对任务的描述，在执行过程中会打印出来。

user：指定调用user模块；

name：user模块里的一个参数，用于指定创建的用户名称

1、创建playbook

```
# cat manage_apache.yml
- name: play to setup web server
  hosts: all
  tasks:
    - name: install latest httpd
      yum:
        name: httpd
        state: latest

    - name: copy index.html
      copy:
        src: files/index.html
        dest: /var/www/html/index.html
```

```
- name: start httpd service
  service:
    name: httpd
    state: started
    enabled: true
```

2、执行playbook

```
# ansible-playbook manage_apache.yml
```

1. 打印详细信息

- ◆ -v : 打印任务运行结果
- ◆ -vv : 打印任务运行结果以及任务的配置信息
- ◆ -vvv : 包含了远程连接的一些信息
- ◆ -vvvv : Adds extra verbosity options to the connection plug-ins, including the users being used in the managed hosts to execute scripts, and what scripts have been executed

2. 校验playbook语法

```
# ansible-playbook --syntax-check manage_apache.yml  
playbook: manage_apache.yml
```

3. 测试运行playbook

通过-C选项可以测试playbook的执行情况，但不会真的执行：

```
# ansible-playbook -C manage_apache.yml
```

Multiple Plays

This is a simple playbook with two plays

- name: first play

hosts: web.example.com

tasks:

- name: first task

yum:

name: httpd

status: present

- name: second task

service:

name: httpd

state: started

- name: second play

hosts: db.example.com

tasks:

- name: first task

yum:

name: mariadb-server

status: present

- name: second task

service:

name: mariadb

state: started

- playbook是由一个或多个"play"组成的列表。play的主要功能就是对一组主机应用play中定义好的task。从根本上来讲一个task就是对ansible一个module的调用。而将多个play按照一定的顺序组织到一个playbook中，我们称之为编排。
- playbook主要有以下四部分构成：
 - ◆ Target section：用于定义将要执行playbook的远程主机组及远程主机组上的用户，还包括定义通过什么样的方式连接远程主机（默认ssh）
 - ◆ Variable section：定义playbook运行时需要使用的变量
 - ◆ Task section：定义将要在远程主机上执行的任务列表
 - ◆ Handler section：定义task执行完成以后需要调用的任务

- playbook中的每一个play的目的都是为了让某个或某些主机以某个指定的用户身份执行任务。
- playbook中的远程用户和ad-hoc中的使用没有区别，默认不定义，则直接使用ansible.cfg配置中的用户相关的配置。也可在playbook中定义如下：

```
- name: create files
hosts: datacenter
remote_user: ansible
become: yes
become_mothod: sudo
become_user: root
tasks:
  - name: create files
    file:
      path: /tmp/redhat
      state: touch
```

Target section

- playbook中的hosts即inventory中的定义主机与主机组，在《Ansible Inventory》中我们讲到了如何选择主机与主机组，在这里也完全适用。

- name: start mariadb

- hosts: db,&london

- tasks:

- name: start mariadb

- service:

- name: mariadb

- state: started

- play的主体部分是任务列表。任务列表中的各任务按次序逐个在hosts中指定的所有主机上执行，在所有主机上完成第一个任务后再开始第二个。在自上而下运行某playbook时，如果中途发生错误，则整个playbook会停止执行，由于playbook的幂等性，playbook可以被反复执行，所以即使发生了错误，在修复错误后，再执行一次即可。

tasks:

- name: make sure apache is running

service:

name: httpd

state: started

- name: disable selinux

command: /sbin/setenforce 0

- 如果命令或脚本的退出码不为零可以使用如下方式替代：

tasks:

- name: run this command and ignore the result

shell: /usr/bin/somecommand || /bin/true

- 可以使用ignore_errors来忽略错误信息：

tasks:

- name: run this command and ignore the result

shell: /usr/bin/somecommand

ignore_errors: True

- 在Ansible Playbook中，handler事实上也是个task，只不过这个task默认并不执行，只有在被触发时才执行。
- handler通过notify来监视某个或者某几个task，一旦task执行结果发生变化，则触发handler，执行相应操作。
- handler会在所有的play都执行完毕之后才会执行，这样可以避免当handler监视的多个task执行结果都发生了变化之后而导致handler的重复执行（handler只需要在最后执行一次即可）。

说明：在notify中定义内容一定要和tasks中定义的 - name 内容一样，这样才能达到触发的效果，否则会不生效。

tasks:

- name: configuration file

copy:

src: config/httpd.conf

dest: /etc/httpd/conf/httpd.conf

notify: **restart apache**

- name: start apache
service

name: httpd

state: started

handlers:

- name: **restart apache**

service:

name: httpd

state: restarted

默认情况下，在一个play中，只要有task执行失败，则play终止，即使是与handler关联的task在失败的task之前运行成功了，handler也不会被执行。如果希望在这种情况下handler仍然能够执行，则需要使用如下配置：

说明：如果与handler关联的task还未执行，在其前的task已经失败，整个play终止，则handler未被触发，也不会执行。

- hosts: all

force_handlers: yes

tasks:

- name: a task which always notifies its handler

command: /bin/true

notify: restart the database

- name: a task which fails because the package doesn't exist

yum:

name: notapkg

state: latest

handlers:

- name: restart the database

service:

name: mariadb

state: restarted

- ✓ Ansible Playbook简介
- ✓ YAML基本语法
- ✓ Playbook基本语法
- ✓ Playbook简单示例
- ✓ ansible-playbook常用选项
- ✓ Multiple Plays
- ✓ playbook的结构说明
- ✓ handler用法

